

Árvores Binárias de Procura



- O elemento na raiz da árvore é maior de todos os elementos na subárvore à esquerda e menor que todos os elementos na subárvore direita
- Ambas as subárvores são árvores binárias de procura

```
data BTree a = Empty
              | Node a (BTree a) (BTree a)
deriving Show
```

```
elemBT :: Ord a => a -> BTree a -> Bool
elemBT _ Empty = false
elemBT x (Node i e d) | x < i = elemBT x e
                      | x > i = elemBT x d
                      | x == i = True
```

$\text{insertBT} :: \text{Ord } a \Rightarrow a \rightarrow \text{BTree } a \rightarrow \text{BTree } a$

$\text{insertBT } n \text{ Empty} = \text{Node } n \text{ Empty Empty}$

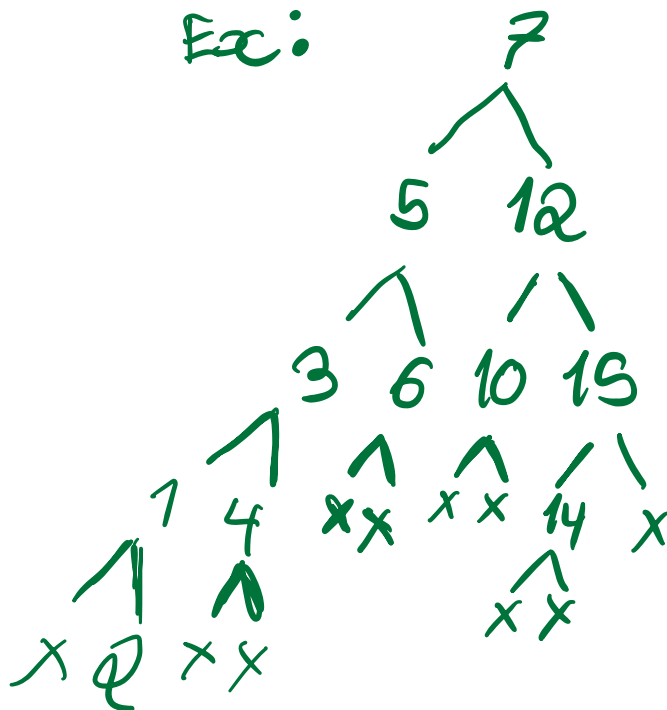
$\text{insertBT } n (\text{Node } i \text{ e d})$

$\quad | n < i = \text{Node } i (\text{insertBT } n \text{ e}) \text{ d}$

$\quad | n > i = \text{Node } i \text{ e} (\text{insertBT } n \text{ d})$

$\quad | n == i = \text{Node } i \text{ e d}$

Ex:



$\text{inorder} \in \rightarrow [1, 2, 3, 4, 5, 6, 7, 10, 12, 14, 15]$

$\text{preorder} \in \rightarrow [7, 5, 3, 1, 2, 4, 6, 12, 10, 15, 14]$

$\text{postorder} \in \rightarrow [2, 1, 4, 3, 6, 5, 10, 14, 15, 12, 7]$

Árvore Binária Balanceada

- ↳ Uma árvore vazia está balanceada
- ↳ A diferença de altura da subárvore esquerda e da direita é no máximo de 1 unidade
- ↳ Ambas as subárvores estão balanceadas

estaBalanceada :: BTree a → Bool

estaBalanceada Empty = True

estaBalanceada (Node e d) =

abs (altura e - altura d) ≤ 1 & &

estaBalanceado e & &

estaBalanceado d

balance BT :: Ord a => BTree a → BTree a

balance BT = consEroi (inorder)

consEroi :: [a] → BTree a

consEroi [] = Empty

consEroi l = let n = length l

m = div n 2

(l1, x, l2) = splitAt m l

in Node x (constroi l_1)(constroi l_2)